CerealeHUB

Sistema Logistico per la Gestione di Spedizioni

Documentazione Tecnica del Progetto

*Lavoro svolto da **Mora Cristian** e **Ceresoli Ettore***

Anno Scolastico 2025/2026

1. Informazioni Generali

Campo	Valore
Titolo del Progetto	CerealeHUB — Sistema Logistico Centrale
Linguaggio	Java
Ambiente di Sviluppo	IntelliJ IDEA

2. Descrizione Sintetica dell'Idea

CerealeHUB è un progetto svolto su Java che simula un sistema logistico per la gestione di pacchi e spedizioni. Il sistema è pensato come una piattaforma centrale che mette in comunicazione tre tipi di attori:

- Utenti: creano e monitorano i propri pacchi da spedire.
- Corrieri: ricevono pacchi assegnati automaticamente e gestiscono le consegne.
- Amministratori (Magazzino): supervisionano l'intero flusso, raccolgono i pacchi dagli utenti e li assegnano ai corrieri disponibili.

Il progetto mette in pratica i concetti fondamentali della programmazione orientata agli oggetti: ereditarietà, polimorfismo, classi astratte, eccezioni personalizzate e l'uso di strutture dati della Java Collection Framework.

3. Struttura Generale del Programma

Il programma è organizzato in tre package principali, più una classe principale e una di gestione:

Package Pacchetti

- **Packet (astratta)** — classe base di tutti i pacchi, implementa Comparable per la priorità.
- **PacketStandard** — estende Packet con tipo STANDARD.
- **PacketPremium** — estende Packet con tipo PREMIUM (ha priorità maggiore nella coda).
- **StatoPacket (enum)** — IN_MAGAZZINO, IN_ATTESA_ASSEGNAZIONE, IN_CONSEGNA, CONSEGNATO.
- **TipoPacket (enum)** — STANDARD, PREMIUM.
- **ErroreNelPacchetto** — eccezione personalizzata per errori sui pacchi.

Package Persone

- **Persona (astratta)** — classe base con nome e cognome.
- **Utente** — estende Persona; gestisce la propria lista di pacchi e le operazioni di creazione.
- **Corriere** — estende Persona; gestisce il furgone con una PriorityQueue di pacchi assegnati.
- **ErroreNellaPersona** — eccezione personalizzata per errori sulle persone.

Classi Principali

- **GestioneMagazzino** — coordina utenti, corrieri e pacchi; contiene la logica centrale del sistema.
- **Main** — punto di ingresso; gestisce i menu interattivi per i tre tipi di utente.

4. Classi Principali Utilizzate

Classe	Responsabilità
Packet	Modello astratto del pacco; gestisce ID, peso, costo, stato, priorità.
PacketPremium / PacketStandard	Concretizzano il tipo di pacco; passano il TipoPacket al costruttore padre.
Persona	Classe astratta con dati anagrafici comuni a Utente e Corriere.
Utente	Crea pacchi tramite scanner, visualizza il proprio storico, espone la lista al magazzino.
Corriere	Riceve pacchi, li gestisce per priorità, esegue consegne e mostra stato del mezzo.
GestioneMagazzino	Raccoglie pacchi dagli utenti, li assegna ai corrieri, tiene uno storico tramite HashMap.
Main	Menu interattivo a tre aree (Utente, Corriere, Admin); gestisce input con try-catch.

5. Collezioni Utilizzate e Motivazione della Scelta

PriorityQueue<Packet>

Usata sia in Utente (lista pacchi personali) che in Corriere (carico del furgone) e in GestioneMagazzino (coda pacchi da assegnare).

La scelta è motivata dal fatto che i pacchi PREMIUM devono sempre essere consegnati prima di quelli STANDARD. Packet implementa Comparable<Packet> con questa logica:

```
PREMIUM → valore 0  (priorità alta)
STANDARD → valore 1  (priorità normale)
```

ArrayList<Utente> e ArrayList<Corriere>

Usate in GestioneMagazzino per memorizzare tutti gli utenti e i corrieri registrati. La scelta ricade su ArrayList perché:

- L'ordine di inserimento è rilevante dato che non ci sono priorità tra utenti e corrieri
- Si cicla velocemente ed è la soluzione più semplice ed effficace

HashMap<String, ArrayList<Packet>>

Usata in GestioneMagazzino come storico delle spedizioni. La chiave è la matricola del corriere, il valore è la lista di pacchi che gli sono stati assegnati. La scelta è motivata da:

- Accesso al carico di un corriere specifico tramite la sua matricola.
- computeIfAbsent() permette di creare la lista al primo accesso in modo pulito.
- È ideale per aggregare dati per chiave (matricola → lista pacchi).

6. Operazioni Principali Realizzate

Creazione di un pacco (Utente)

L'utente inserisce peso, costo, tipo (Standard/Premium) e se il pacco è fragile. Il costruttore di Packet applica automaticamente le regole di prezzo: +10€ se il peso supera 50 kg, -5€ se il tipo è PREMIUM.

Raccolta pacchi dal magazzino

GestioneMagazzino.raccogliPacchiDaUtenti() scorre tutti gli utenti, copia le loro PriorityQueue e aggiunge a pacchiDaConsegnare solo i pacchi con stato IN_MAGAZZINO.

Assegnazione pacchi ai corrieri (Round-Robin)

GestioneMagazzino.assegnaPacchi() distribuisce i pacchi ai corrieri a turno. Se un corriere ha il furgone pieno (capacità massima: 15 pacchi), viene saltato. L'algoritmo conta i tentativi falliti e si ferma se tutti i corrieri sono pieni, stampando un avviso con il numero di pacchi rimasti.

Consegna di un pacco (Corriere)

Il corriere esegue poll() sulla sua PriorityQueue: viene estratto il pacco con priorità più alta (PREMIUM prima di STANDARD). Lo stato viene aggiornato a CONSEGNATO e viene mostrato il dettaglio del pacco e lo spazio rimanente nel furgone.

Matricola automatica dei corrieri

Ogni nuovo Corriere riceve una matricola generata automaticamente nel formato AA01, AA02, ..., AA99, AB01, ecc. Il contatore statico garantisce l'unicità tra istanze.

7. Difficoltà Incontrate e Soluzioni Adottate

Difficoltà	Soluzione Adottata
raccogliPacchiDaUtenti() rimuoveva i pacchi pure dalla lista dell'utente tramite poll	Creata una copia della lista, dato che poll è fondamentale per poter far rispettare la priorityQueue
Crash con InputMismatchException nei menu	Tutti i nextInt() nei menu sono racchiusi in try-catch; in caso di input errato si pulisce il buffer e si ripete il ciclo.
Uso dell'hashmap nello storico per gestire tutto in modo efficiente	Aggiunto metodo computeifabsent che se non trova la chiave crea la chiave con la lista, invece se esiste si limita ad aggiungere il pacchetto
Ciclo infinito in assegnaPacchi	Aggiunta variabile tentativiFalliti affinchè che non si entri in loop infiniti quando tutti i corrieri sono occupati ed una volta che li ha passati tutti esca dal ciclo while

Fine della Documentazione — CerealeHUB v1.0